

Solving SAT

1 What Is SAT? Starting from Scratch

Imagine you have a set of Boolean (true/false) variables and a collection of rules about them. The question “Is there a way to set the variables so that *all* the rules are satisfied at once?” is the **Boolean Satisfiability Problem**, or SAT.

SAT is important for two reasons. First, it is the *first* problem proven to be NP-complete (Cook, 1971) — if we could solve it efficiently, thousands of other hard problems would become easy too. Second, despite being NP-complete in the worst case, modern SAT *solvers* handle industrial instances with *millions* of variables every day. Understanding why requires the algorithms in this document.

1.1 Variables, Literals, Clauses, and CNF

Definition 1.1: Vocabulary of SAT

- A **variable** x takes value **T** (true) or **F** (false).
- A **literal** is either a variable x (positive literal, satisfied when $x = \mathbf{T}$) or its negation $\neg x$ (negative literal, satisfied when $x = \mathbf{F}$).
- A **clause** is a set of literals connected by OR.
Example: $(x_1 \vee \neg x_2 \vee x_3)$. A clause is satisfied if *at least one* of its literals is **T**.
- A formula in **Conjunctive Normal Form (CNF)** is a set of clauses connected by AND.
Example: $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$. The formula is satisfied if *every* clause is satisfied.

Definition 1.2: Special clause types

- **Unit clause**: a clause with exactly one unset literal. That literal *must* be set to **T**, otherwise this clause cannot be satisfied.
- **Empty clause**: a clause whose every literal has been set to **F**. This clause can *never* be satisfied, so any formula containing an empty clause is **UNSAT** (unsatisfiable).

Example 1.1: A small CNF formula

Let $\varphi = \underbrace{(x_1 \vee x_2)}_{C_1} \wedge \underbrace{(\neg x_1 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_2 \vee \neg x_3)}_{C_3}$.

Setting $x_1 = \mathbf{T}$, $x_2 = \mathbf{F}$, $x_3 = \mathbf{T}$ satisfies all three clauses: C_1 via $x_1 = \mathbf{T}$; C_2 via $x_3 = \mathbf{T}$; C_3 via $\neg x_2 = \mathbf{T}$. So φ is **SAT**.

1.2 What Is k-SAT?

When every clause has *exactly* k literals, the problem is called k -SAT. Two cases stand out:

- **2-SAT**: solvable in *polynomial time* (Section 8).
- **3-SAT** (and k -SAT for $k \geq 3$): NP-complete.

The difference between 2-SAT and 3-SAT is one of the most striking examples of how a tiny change in a problem's rules can change its difficulty from easy to (apparently) hard.

2 The Naïve Approach and Why We Need Better Ideas

The most obvious algorithm tries every possible assignment of T/F to the n variables. There are 2^n assignments. For $n = 30$ that is already about 10^9 — a second's work for a modern computer. But for $n = 100$ it is 10^{30} , which would take longer than the age of the universe.

The fundamental idea behind every practical SAT algorithm is **backtracking with pruning**:

1. Assign variables one at a time.
2. After each assignment, check whether the partial assignment is already doomed to fail.
3. If it is, *backtrack*: undo the last decision and try the other value, instead of exploring the entire subtree.

3 Clause-Guided Branching

3.1 The Core Observation

Key Idea 3.1: Unsatisfied clauses tell us what to do

Consider any unsatisfied clause $C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$. Any satisfying assignment *must* set at least one of $\ell_1, \dots, \ell_k$ to T.

So instead of guessing which variable to set next, we can use the first unsatisfied clause as a *guide*: branch by trying $\ell_1 = \text{T}$, then $\ell_2 = \text{T}$, and so on, until one branch succeeds. If none succeed, the formula is UNSAT.

This is much better than random guessing: we know that *at least one* of the at-most- k branches must be correct.

3.2 Setting Variables One at a Time

A critical refinement is to set variables **one at a time** and *simplify the formula immediately* after each assignment:

1. Remove every clause that is now satisfied (it cannot be violated anymore).
2. Remove any falsified literal from every remaining clause.
3. Check the simplified formula: has a unit clause appeared? A unit clause forces the next assignment for free.

Example 3.1: Step-by-step simplification

$$\varphi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3).$$

Decision: set $x_1 = \mathbf{T}$.

- $C_1 = (x_1 \vee x_2 \vee \neg x_3)$: $x_1 = \mathbf{T}$ satisfies it. *Remove* C_1 .
- $C_2 = (\neg x_1 \vee x_3)$: $\neg x_1$ is now $\mathbf{F}$, remove it. C_2 shrinks to the unit clause (x_3) .

Forced by unit clause: (x_3) forces $x_3 = \mathbf{T}$.

- $C_3 = (\neg x_2 \vee \neg x_3)$: $\neg x_3$ is now $\mathbf{F}$, remove it. C_3 shrinks to the unit clause $(\neg x_2)$.

Forced by unit clause: $(\neg x_2)$ forces $x_2 = \mathbf{F}$.

No clauses remain. **Solution:** $x_1 = \mathbf{T}$, $x_2 = \mathbf{F}$, $x_3 = \mathbf{T}$.

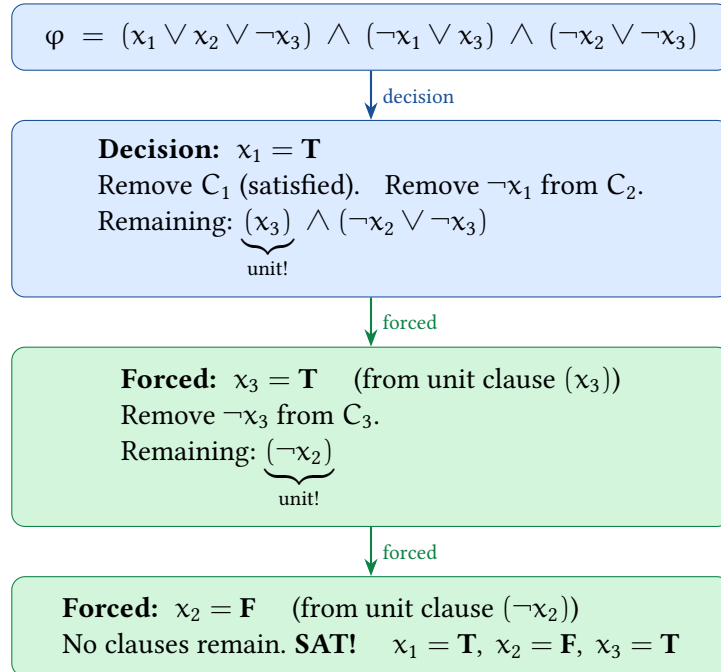


Figure 1: Step-by-step simplification. One decision triggers two forced assignments through unit clauses.

4 Unit Propagation and Resolution

4.1 Unit Propagation

The process of repeatedly following unit clauses has a name.

Definition 4.1: Unit Propagation

Unit propagation (also called Boolean Constraint Propagation, BCP) repeatedly applies the following rule until no unit clauses remain:

If (ℓ) is a unit clause, set $\ell = \mathbf{T}$ and simplify.

A **conflict** occurs if the formula contains an empty clause after simplification.

Unit propagation can trigger a **cascade**: one forced assignment creates a new unit clause, which forces another assignment, and so on.

Example 4.1: A propagation cascade

$$\varphi = (x_1) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3)$$

No decisions needed. Unit propagation alone handles everything:

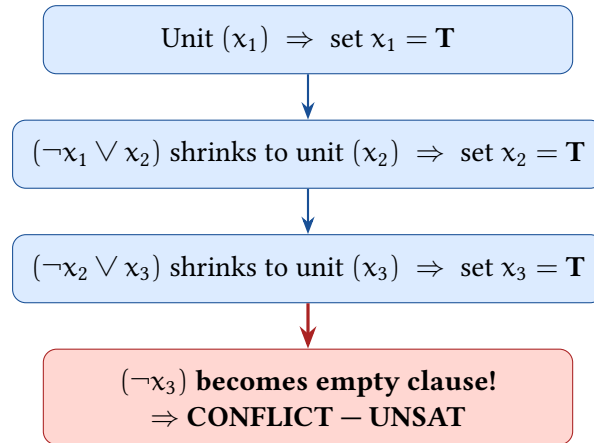


Figure 2: A unit propagation cascade reveals that the formula is unsatisfiable without any branching decisions.

4.2 The Resolution Rule

Unit propagation is a special case of a general logical rule called **resolution**.

Definition 4.2: The Resolution Rule

Given two clauses sharing variable x in opposite signs:

$$C_1 = (A \vee x) \quad \text{and} \quad C_2 = (B \vee \neg x)$$

the **resolvent** is $(A \vee B)$. We write $C_1 \otimes_x C_2 = (A \vee B)$.

Why is the resolvent valid? If $x = \mathbf{T}$: clause C_2 requires $B = \mathbf{T}$. If $x = \mathbf{F}$: clause C_1 requires $A = \mathbf{T}$. Either way, $(A \vee B)$ must hold. So $(A \vee B)$ is a logical consequence of $C_1 \wedge C_2$.

Connection to unit propagation: When $C_1 = (x)$ and $C_2 = (B \vee \neg x)$, the resolvent is (B) , which is exactly what unit propagation produces: remove $\neg x$ from C_2 .

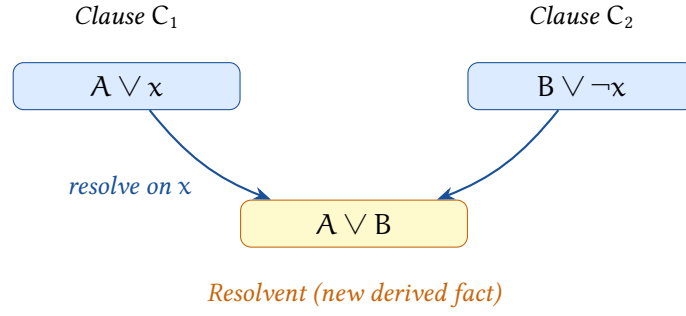


Figure 3: Resolution: eliminate variable x by combining two parent clauses. The resolvent is a new clause that follows logically from both parents.

5 Implication Graphs: Tracking Why Variables Are Set

5.1 Motivation

As unit propagation runs, it creates a web of cause and effect: decision A forces B , which forces C , and eventually causes a conflict. An **implication graph** makes this causal structure *visible*, allowing us to pinpoint *exactly* which decisions caused the conflict.

This is not just bookkeeping. Reading the implication graph lets us derive **learnt clauses** that prevent the solver from falling into the same trap again, which is the key idea behind modern solvers (CDCL, Section 7).

5.2 Building an Implication Graph

Definition 5.1: Implication Graph

An **implication graph** is a directed graph built as unit propagation runs:

- **Nodes:** one node per assigned literal, labelled with the literal and its **decision level** (how many branching decisions have been made so far, counting from 0).
- **Decision nodes** (drawn with double border): literals we chose freely, not forced. They have no incoming edges.
- **Implication edges:** if setting literals $\ell_1, \dots, \ell_{k-1}$ to **F** forced literal ℓ_k to **T** via clause $C = (\ell_1 \vee \dots \vee \ell_{k-1} \vee \ell_k)$, draw an arrow from each ℓ_i (labelled C) to ℓ_k . In words: “ ℓ_i contributed to forcing ℓ_k .”
- **Conflict node** κ (drawn as a diamond): when a clause C becomes empty, draw arrows from each literal in C to κ , labelled C .

Building the graph step by step

The process is straightforward. For each implied literal, ask: “which clause became a unit and forced this?” Draw an arrow from each other literal of that clause (which were already set to **F**) to the forced literal.

Example 5.1: Building an implication graph from scratch

Formula:

$$C_1 = (x_1 \vee x_2)$$

$$C_2 = (\neg x_1 \vee x_3)$$

$$C_3 = (\neg x_2 \vee x_3)$$

$$C_4 = (\neg x_3 \vee x_4)$$

$$C_5 = (\neg x_3 \vee \neg x_4)$$

Decision at level 1: set $x_1 = \mathbf{T}$.

- $C_2 = (\neg x_1 \vee x_3)$: $\neg x_1$ is **F**. Unit clause (x_3). **Force** $x_3 = \mathbf{T}$. Draw edge: $x_1 \rightarrow x_3$ (labelled C_2).
- $C_4 = (\neg x_3 \vee x_4)$: $\neg x_3$ is **F**. Unit clause (x_4). **Force** $x_4 = \mathbf{T}$. Draw edge: $x_3 \rightarrow x_4$ (labelled C_4).
- $C_5 = (\neg x_3 \vee \neg x_4)$: both $\neg x_3 = \mathbf{F}$ and $\neg x_4 = \mathbf{F}$. **Empty clause! Conflict**. Draw edges $x_3 \rightarrow \kappa$ and $x_4 \rightarrow \kappa$ (labelled C_5).

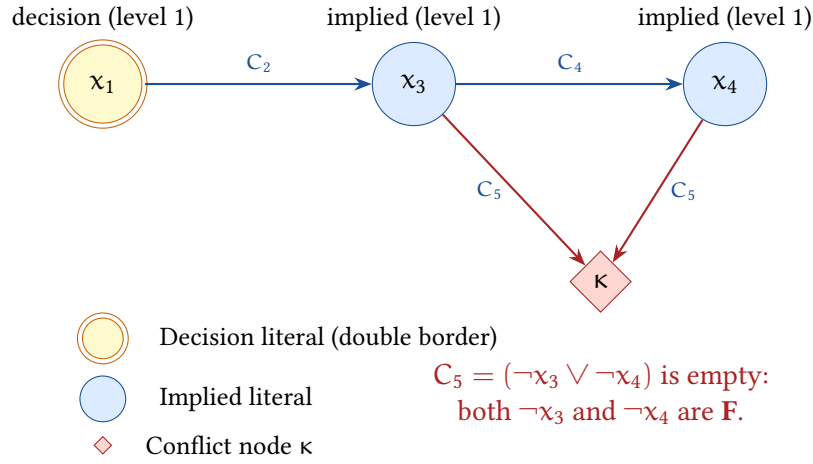


Figure 4: Implication graph for Example 5.1. Decision x_1 (level 1) forces x_3 via C_2 , which forces x_4 via C_4 . Both x_3 and x_4 then conflict via C_5 .

5.3 A Richer Example: Multiple Decision Levels

Usually we make several decisions before hitting a conflict. Each decision is a new level, and the implication graph grows.

Example 5.2: Two decision levels

Formula (C_1 through C_6):

$$C_1 = (x_1 \vee x_2)$$

$$C_2 = (\neg x_1 \vee x_3 \vee x_5)$$

$$C_3 = (\neg x_2 \vee x_4 \vee x_5)$$

$$C_4 = (\neg x_3 \vee \neg x_4 \vee x_6)$$

$$C_5 = (\neg x_5 \vee x_6)$$

$$C_6 = (\neg x_6)$$

Level 1: Decide $x_1 = \mathbf{T}$ and $x_2 = \mathbf{T}$. No unit clauses are created yet (e.g. C_2 still has two unset literals).

Level 2: Decide $x_5 = \mathbf{F}$.

- $C_2 = (\neg x_1 \vee x_3 \vee x_5)$: $\neg x_1 = \mathbf{F}$ and $x_5 = \mathbf{F}$. Unit: (x_3) . **Force** $x_3 = \mathbf{T}$. Edges: $x_1 \rightarrow x_3, \neg x_5 \rightarrow x_3$ (labelled C_2).
- $C_3 = (\neg x_2 \vee x_4 \vee x_5)$: $\neg x_2 = \mathbf{F}$ and $x_5 = \mathbf{F}$. Unit: (x_4) . **Force** $x_4 = \mathbf{T}$. Edges: $x_2 \rightarrow x_4, \neg x_5 \rightarrow x_4$ (labelled C_3).
- $C_4 = (\neg x_3 \vee \neg x_4 \vee x_6)$: $\neg x_3 = \mathbf{F}$ and $\neg x_4 = \mathbf{F}$. Unit: (x_6) . **Force** $x_6 = \mathbf{T}$. Edges: $x_3 \rightarrow x_6, x_4 \rightarrow x_6$ (labelled C_4).
- $C_6 = (\neg x_6)$: $\neg x_6 = \mathbf{F}$. **Empty clause! Conflict.** Edge: $x_6 \rightarrow \kappa$ (labelled C_6).

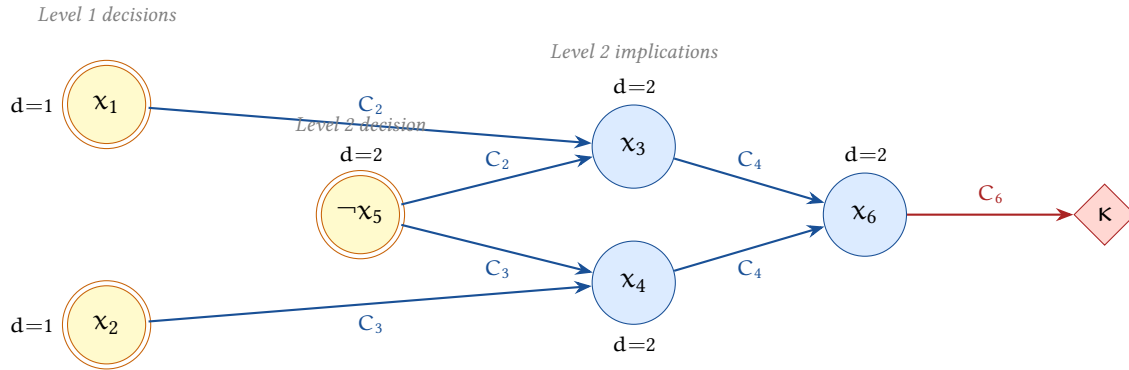


Figure 5: Two-level implication graph for Example 5.2. Level-1 decisions x_1, x_2 and level-2 decision $\neg x_5$ together force x_3, x_4, x_6 , which then conflict via C_6 .

5.4 Reading Causes from the Graph: Cuts and Learnt Clauses

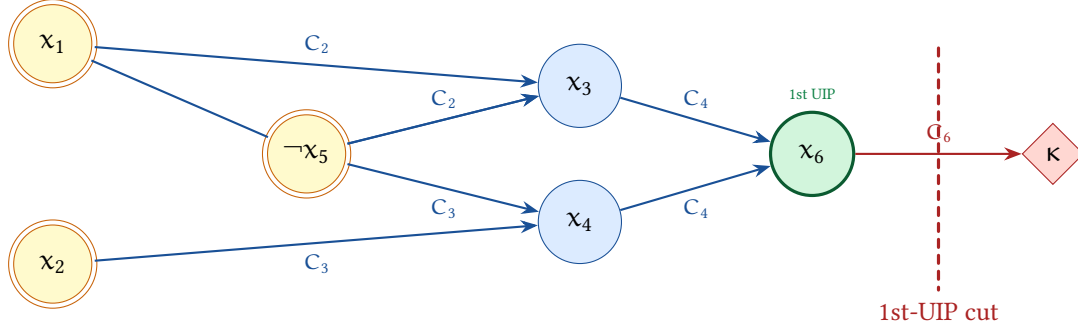
The implication graph lets us answer: *which decisions caused the conflict?* We find the answer by drawing a **cut**.

Key Idea 5.1: Learnt Clause from a Cut

Draw a directed cut that separates the conflict node κ (on the right) from some set of nodes on the left. Let the **reason set** be the set of nodes immediately to the left of the cut that have edges crossing it.

The **learnt clause** is the disjunction of the *negations* of all literals in the reason set. Intuitively: “these literals cannot all be **T** simultaneously.”

Adding the learnt clause to the formula prevents the solver from repeating the same conflict.



Reason set = $\{x_6\}$. Learnt clause = $(\neg x_6)$.
Backjump to level 1.

Figure 6: The 1st-UIP cut in Example 5.2. Only x_6 crosses the cut from left to right. The learnt clause is $(\neg x_6)$.

What is the First UIP?

A **Unique Implication Point** (UIP) at the current decision level is any node that lies on *every* path from the current decision literal to the conflict node κ . The **First UIP** is the one closest to κ . Cutting at the First UIP typically gives the shortest and most useful learnt clause.

6 DPLL: The Classical SAT Algorithm

6.1 The Algorithm

DPLL (Davis, Putnam, Logemann, Loveland, 1962) packages backtracking, unit propagation, and one more useful rule into a clean procedure.

Definition 6.1: Pure Literal Elimination

A literal ℓ is **pure** if its negation $\neg\ell$ does not appear anywhere in the formula. Since $\neg\ell$ is absent, we can always safely set $\ell = \mathbf{T}$: every clause containing ℓ is satisfied, and no clause is harmed.

Algorithm 1 DPLL

```

1: procedure DPLL( $\varphi$ )
2:   Apply unit propagation and pure literal elimination exhaustively
3:   if no clauses remain then return SAT
4:   end if
5:   if an empty clause exists then return UNSAT
6:   end if
7:   Choose an unset variable  $x$ 
8:   if DPLL( $\varphi$  with  $x = \mathbf{T}$ ) = SAT then return SAT
9:   end if return DPLL( $\varphi$  with  $x = \mathbf{F}$ )
10: end procedure

```

Example 6.1: DPLL on a 4-variable formula

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$$

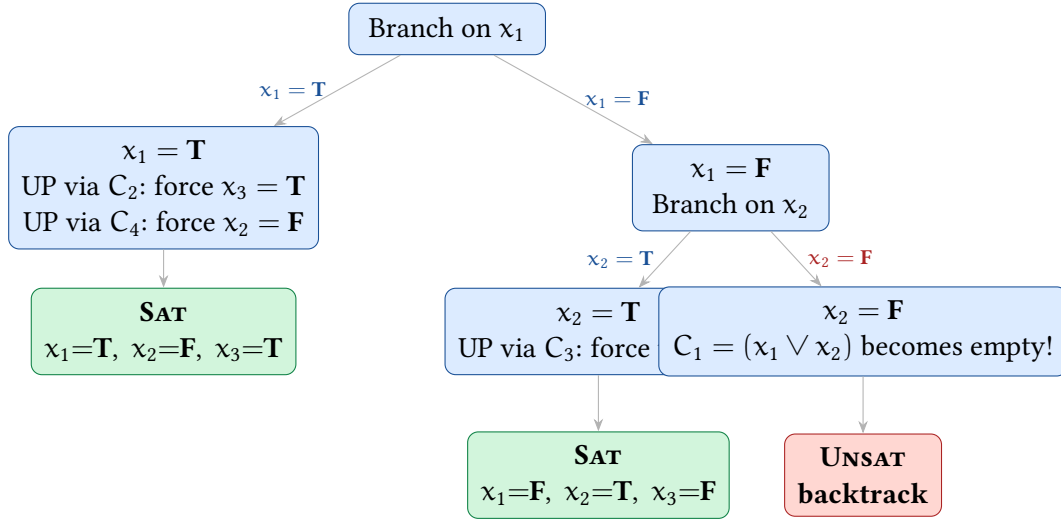


Figure 7: DPLL search tree. The left branch ($x_1 = \mathbf{T}$) immediately triggers two forced assignments and finds a solution without further branching.

7 CDCL: Learning from Conflicts

7.1 The Problem with DPLL

When DPLL hits a dead end, it undoes the most recent decision. But the conflict may have been caused by a decision made *many levels earlier*. DPLL wastes time exploring variations of recent decisions that cannot possibly fix the underlying problem.

Key Idea 7.1: What CDCL Adds

1. **Conflict analysis:** use the implication graph to find the root causes of the conflict.
2. **Clause learning:** add a learnt clause encoding the fact that the root causes cannot all be $\mathbf{T}$ simultaneously. This prevents the same conflict from recurring.
3. **Non-chronological backjumping:** jump back directly to the decision level identified by the learnt clause, skipping all intermediate levels.

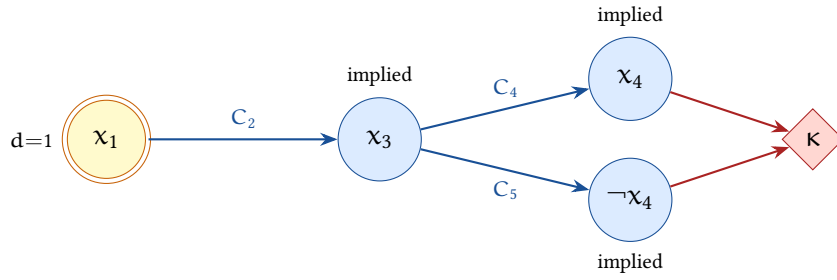
Algorithm 2 CDCL

```
1: procedure CDCL( $\varphi$ )
2:    $d \leftarrow 0$ 
3:   Run unit propagation; if conflict: return UNSAT
4:   while not all variables assigned do
5:     Choose variable  $x$  and value  $v$ ; assign  $x \leftarrow v$  at level  $d+1$ ;  $d++$ 
6:     while unit propagation finds a conflict do
7:       if  $d = 0$  then return UNSAT
8:       end if
9:       Analyse implication graph: derive learnt clause  $C_\ell$  and backjump level  $d'$ 
10:      Add  $C_\ell$  to  $\varphi$ ; undo assignments above level  $d'$ ;  $d \leftarrow d'$ 
11:    end while
12:  end while return SAT
13: end procedure
```

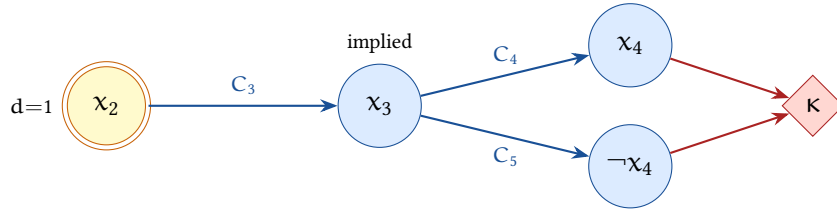
Example 7.1: CDCL detecting UNSAT

$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_3 \vee \neg x_4)$.

Notice that C_4 and C_5 force x_4 and $\neg x_4$ whenever $x_3 = \mathbf{T}$. So x_3 can never be $\mathbf{T}$. But C_2 and C_3 show that both $x_1 = \mathbf{T}$ and $x_2 = \mathbf{T}$ force $x_3 = \mathbf{T}$. So neither x_1 nor x_2 can be $\mathbf{T}$. But then C_1 is unsatisfied. **CDCL discovers this automatically.**



Learnt clause: $(\neg x_1)$.
Force $x_1 = \mathbf{F}$. Try $x_2 = \mathbf{T}$.



Learnt clause: $(\neg x_2)$.
Force $x_2 = \mathbf{F}$. $C_1 = (x_1 \vee x_2)$: both $\mathbf{F}$. **UNSAT**.

Figure 8: CDCL on Example 7.1. *Top*: deciding $x_1 = \mathbf{T}$ immediately leads to a conflict; learnt clause $(\neg x_1)$ forces $x_1 = \mathbf{F}$. *Bottom*: deciding $x_2 = \mathbf{T}$ leads to the same conflict; learnt clause $(\neg x_2)$ forces $x_2 = \mathbf{F}$, making C_1 empty and proving UNSAT.

8 2-SAT: A Polynomial-Time Algorithm

8.1 Why 2-SAT is Special

In 2-SAT every clause has exactly two literals. A two-literal clause $(a \vee b)$ says: “at least one of a or b must be $\mathbf{T}$.” We can rewrite this as two *if-then* implications:

$$(a \vee b) \iff (\neg a \Rightarrow b) \wedge (\neg b \Rightarrow a).$$

Why? If $a = \mathbf{F}$ (i.e. $\neg a = \mathbf{T}$), then b must be $\mathbf{T}$ to satisfy the clause. Conversely, if $b = \mathbf{F}$, then a must be $\mathbf{T}$.

This means the entire 2-SAT formula can be represented as a *directed graph of implications*. Satisfiability then becomes a graph-reachability question, solvable in linear time.

8.2 The 2-SAT Implication Graph

Definition 8.1: 2-SAT Implication Graph

Given a 2-SAT formula φ with variables $x_1, \dots, x_n$, build a directed graph G_φ as follows:

- **Nodes:** $2n$ nodes, one for each literal: $x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$.
- **Edges:** for each clause $(\alpha \vee \beta)$, add two edges:

$$\neg\alpha \longrightarrow \beta \quad \text{and} \quad \neg\beta \longrightarrow \alpha.$$

An edge $u \rightarrow v$ means: “if $u = \mathbf{T}$ then v must be $\mathbf{T}$.”

Example 8.1: Building the 2-SAT implication graph

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

Extract two implications from each clause:

Clause	Implication 1	Implication 2
$(x_1 \vee x_2)$	$\neg x_1 \rightarrow x_2$	$\neg x_2 \rightarrow x_1$
$(\neg x_1 \vee x_3)$	$x_1 \rightarrow x_3$	$\neg x_3 \rightarrow \neg x_1$
$(\neg x_2 \vee \neg x_3)$	$x_2 \rightarrow \neg x_3$	$x_3 \rightarrow \neg x_2$

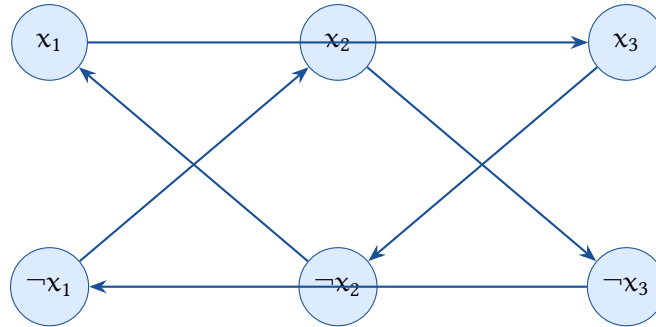


Figure 9: 2-SAT implication graph for Example 8.1. Each of the three clauses contributes exactly two directed edges.

8.3 Why Strongly Connected Components Matter

Definition 8.2: Strongly Connected Component (SCC)

A **Strongly Connected Component** of a directed graph is a maximal set of nodes S such that for *every* pair $u, v \in S$, there is a directed path from u to v and from v to u in the graph.

Every directed graph can be partitioned into SCCs in $O(n + m)$ time using Tarjan’s or Kosaraju’s algorithm.

Key Idea 8.1: The 2-SAT satisfiability condition

A 2-SAT formula is **SAT** if and only if no variable x_i and its negation $\neg x_i$ belong to the *same* SCC.

Why UNSAT when they share an SCC?

If x_i and $\neg x_i$ are in the same SCC, there is a path $x_i \rightsquigarrow \neg x_i$ in the graph. This path represents a chain of implications: “if $x_i = \mathbf{T}$, then ..., then $\neg x_i = \mathbf{T}$.” But $\neg x_i = \mathbf{T}$ means $x_i = \mathbf{F}$ – a contradiction. Similarly, the return path $\neg x_i \rightsquigarrow x_i$ means setting $x_i = \mathbf{F}$ also leads to a contradiction. So x_i can be neither **T** nor **F**, which is impossible.

Why SAT when they are in different SCCs?

If no variable shares an SCC with its negation, we can always construct a valid assignment using the condensation DAG (see Section 8.4).

Example 8.2: SCCs and satisfiability

$$\varphi = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_3 \vee \neg x_1)$$

Implication edges from each clause:

- $(x_1 \vee \neg x_2)$: $\neg x_1 \rightarrow \neg x_2$ and $x_2 \rightarrow x_1$.
- $(\neg x_1 \vee x_2)$: $x_1 \rightarrow x_2$ and $\neg x_2 \rightarrow \neg x_1$.
- $(x_2 \vee x_3)$: $\neg x_2 \rightarrow x_3$ and $\neg x_3 \rightarrow x_2$.
- $(\neg x_3 \vee \neg x_1)$: $x_3 \rightarrow \neg x_1$ and $x_1 \rightarrow \neg x_3$.

Notice: $x_1 \rightarrow x_2$ and $x_2 \rightarrow x_1$, so x_1 and x_2 are in the same SCC. Similarly $\neg x_1 \rightarrow \neg x_2$ and $\neg x_2 \rightarrow \neg x_1$.

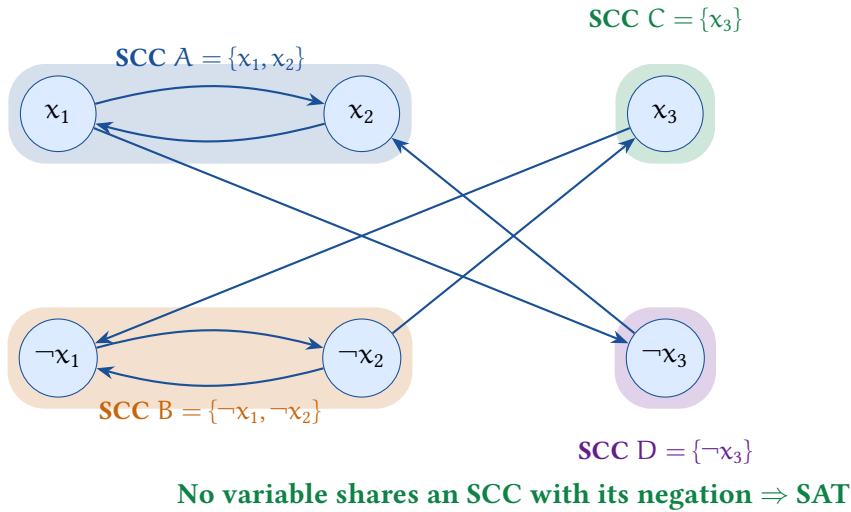


Figure 10: Implication graph for Example 8.2 with SCCs shaded. x_1 and $\neg x_1$ are in different SCCs (A vs B); same for x_2 and x_3 . Formula is satisfiable.

Example 8.3: An unsatisfiable 2-SAT instance

$$\varphi = (x_1) \wedge (\neg x_1). \text{ (Equivalently, } (x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1). \text{)}$$

Implications: $\neg x_1 \rightarrow x_1$ (from the first) and $x_1 \rightarrow \neg x_1$ (from the second). So x_1 and $\neg x_1$ can each reach the other: they are in the *same* SCC. **UNSAT**.

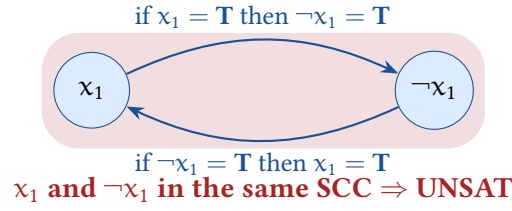


Figure 11: The implication graph for $(x_1) \wedge (\neg x_1)$ has a cycle between x_1 and $\neg x_1$, placing them in the same SCC.

8.4 Extracting a Satisfying Assignment

Once we know the formula is satisfiable, we extract an assignment from the SCC structure.

Key Idea 8.2: Assignment via Condensation DAG

The **condensation** of the implication graph is a new directed graph where each node is an SCC and there is an edge from SCC S to SCC T if any node in S has an edge to any node in T . The condensation is always a **DAG** (directed acyclic graph), and has a topological order.

Assignment rule: For each variable x_i , set

$$x_i = \mathbf{T} \quad \text{if SCC containing } x_i \text{ is later (topologically) than SCC containing } \neg x_i.$$

Otherwise set $x_i = \mathbf{F}$.

Intuition: if $\neg x_i$ comes first topologically, then x_i appears “downstream” and is a safer choice for **T** – setting it to **T** does not risk forcing $\neg x_i$.

Example 8.4: Reading off the assignment

From Example 8.2, the four SCCs are: $A = \{x_1, x_2\}$, $B = \{\neg x_1, \neg x_2\}$, $C = \{x_3\}$, $D = \{\neg x_3\}$. Condensation edges: $A \rightarrow B$ (from $x_1 \rightarrow \neg x_3 \leftarrow \dots$), $A \rightarrow D$, $B \rightarrow C$, $D \rightarrow C$. One topological order: A, D, B, C .

- x_1 : SCC A is before SCC B , so $\neg x_1$ (in B) is later. Set $x_1 = \mathbf{F}$.
- x_2 : same SCC as x_1 . Set $x_2 = \mathbf{F}$.
- x_3 : SCC C is after SCC D . Set $x_3 = \mathbf{T}$.

Verify $x_1 = \mathbf{F}$, $x_2 = \mathbf{F}$, $x_3 = \mathbf{T}$:

- $(x_1 \vee \neg x_2) = (\mathbf{F} \vee \mathbf{T}) = \mathbf{T}$. ✓
- $(\neg x_1 \vee x_2) = (\mathbf{T} \vee \mathbf{F}) = \mathbf{T}$. ✓
- $(x_2 \vee x_3) = (\mathbf{F} \vee \mathbf{T}) = \mathbf{T}$. ✓
- $(\neg x_3 \vee \neg x_1) = (\mathbf{F} \vee \mathbf{T}) = \mathbf{T}$. ✓

8.5 The Full 2-SAT Algorithm and its Complexity

Algorithm 3 2-SAT

```

1: procedure 2-SAT( $\varphi$ )
2:   Build implication graph  $G_\varphi$  ▷  $2n$  nodes,  $2m$  edges
3:   Compute SCCs of  $G_\varphi$  ▷ Tarjan's or Kosaraju's:  $O(n + m)$ 
4:   for each variable  $x_i$  do
5:     if  $\text{scc}(x_i) = \text{scc}(\neg x_i)$  then return UNSAT
6:     end if
7:   end for
8:   for each variable  $x_i$  do ▷ read off assignment
9:     if  $\text{scc}(\neg x_i)$  precedes  $\text{scc}(x_i)$  topologically then
10:       $x_i \leftarrow \mathbf{T}$ 
11:    else
12:       $x_i \leftarrow \mathbf{F}$ 
13:    end if
14:   end for return SAT
15: end procedure

```

Theorem 8.1: 2-SAT runs in linear time

The 2-SAT algorithm solves any 2-SAT instance with n variables and m clauses in $O(n + m)$ time:

- Building G_φ : two edges per clause $\Rightarrow O(m)$.
- Computing SCCs: Tarjan's algorithm runs in $O(n + m)$.
- Reading off the assignment: $O(n)$.

This is **linear time** — as fast as reading the input. The contrast with 3-SAT (NP-complete, no polynomial algorithm known) is remarkable: two literals per clause $\Rightarrow$ easy; three $\Rightarrow$ (apparently) hard.

9 Summary and Comparison

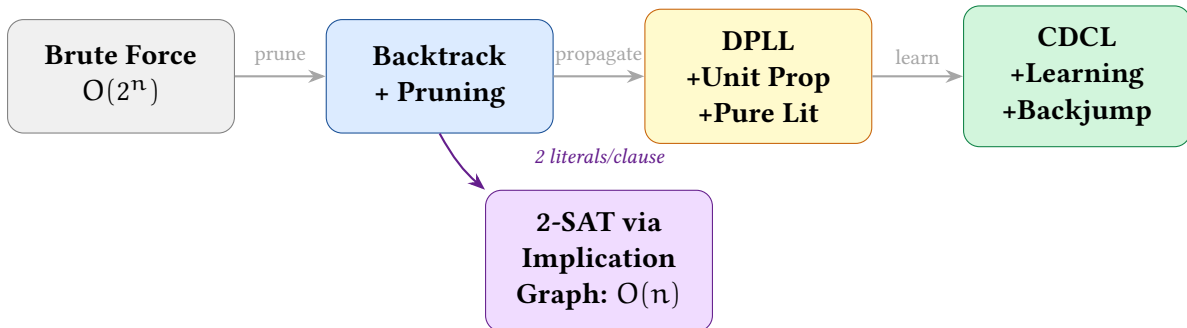


Figure 12: The SAT algorithm family. Each step adds a new technique on top of the previous one. 2-SAT is a special case solvable in linear time.

Feature	Brute Force	Backtrack	DPLL	CDCL	2-SAT
Clause-guided branching	—	✓	✓	✓	—
Unit propagation	—	—	✓	✓	implicit
Pure literal elim.	—	—	✓	optional	—
Clause learning	—	—	—	✓	—
Non-chron. backjump	—	—	—	✓	—
Implication graph	—	—	—	✓	✓
Polynomial time	—	—	—	—	✓

10 Exercises

1. **(Simplification)** Apply one decision and then simplify exhaustively (using unit propagation) on:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_3).$$

Try the decision $x_1 = \mathbf{T}$. What variables are forced? Is the result SAT or UNSAT?

2. **(Unit propagation)** Apply unit propagation exhaustively to:

$$(\neg x_1) \wedge (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2 \vee x_3).$$

What formula remains after propagation?

3. **(Resolution)** Using only the resolution rule, derive the empty clause from $\{(x), (\neg x \vee y), (\neg y)\}$. Show each step.

4. **(Implication graph)** Draw the implication graph for

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3)$$

after the decision $x_1 = \mathbf{T}$. Identify the conflict node and state the learnt clause.

5. **(DPLL)** Run DPLL on

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3).$$

Draw the full search tree showing all decisions and unit propagations.

6. **(2-SAT: building the graph)** Convert each clause of

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_1)$$

into two implications. Draw the implication graph.

7. **(2-SAT: SCCs)** Find all SCCs of the graph from Exercise 6. Is the formula satisfiable? If so, extract an assignment using the condensation DAG and verify it.

8. **(2-SAT: UNSAT)** Show that

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2)$$

is UNSAT using the SCC method.

9. **(Why 3-SAT differs)** A 3-SAT clause $(a \vee b \vee c)$ would translate to $\neg a \Rightarrow (b \vee c)$. Explain in your own words why “ $(b \vee c)$ ” on the right-hand side prevents the 2-SAT polynomial trick from working for 3-SAT.

10. **(Challenge)** Prove that if the 2-SAT implication graph contains both a path $x_i \rightsquigarrow \neg x_i$ and a path $\neg x_i \rightsquigarrow x_i$, then *every* truth assignment violates at least one clause.

Summary of Key Ideas

What to Remember from Each Section

1. **Clauses as guides:** use the first unsatisfied clause to decide which variable to set next.
2. **Unit propagation:** one decision can cascade into many forced assignments.
3. **Resolution:** the logical foundation explaining why unit propagation is sound.
4. **Implication graphs:** visualise the cause-and-effect chain of forced assignments; the graph reveals *why* a conflict happened.
5. **DPLL:** backtracking + unit propagation + pure literal elimination. Complete and simple.
6. **CDCL:** DPLL + clause learning from implication graphs + non-chronological backjumping. Powers all modern SAT solvers.
7. **2-SAT:** every 2-literal clause is two implications. $\text{SAT} \Leftrightarrow$ no variable shares an SCC with its negation. Solvable in $O(n + m)$ time.